

第5章 树

树是一种数据结构，也是逻辑上的。树在计算机领域应用非常广泛。

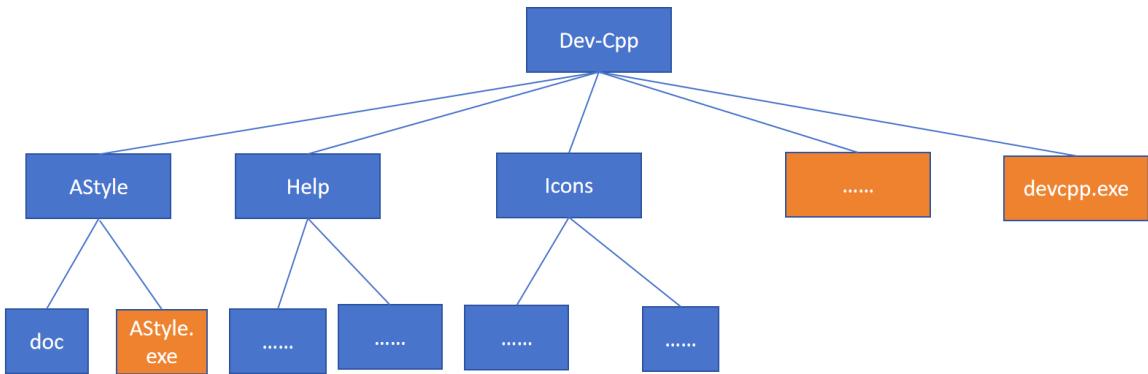
一、树的案例

1.1 文件夹

此电脑 > 本地磁盘 (D:) > Users > Dev-Cpp >

名称	修改日期	类型	大小
AStyle	2024-07-09 20:35	文件夹	
Help	2024-07-09 20:35	文件夹	
Icons	2024-07-09 20:35	文件夹	
Lang	2024-07-09 20:35	文件夹	
MinGW64	2024-07-09 20:35	文件夹	
Templates	2024-07-09 20:35	文件夹	
ConsolePauser.exe	2015-03-22 20:08	应用程序	167 KB
COPYING.txt	2015-03-26 2:17	文本文档	19 KB
devcpp.exe	2015-04-27 22:11	应用程序	2,730 KB
devcpp.exe.manifest	2011-08-23 14:07	MANIFEST 文件	1 KB
devcpp.map	2015-04-27 22:11	MAP 文件	1,543 KB
devcppPortable.exe	2015-03-22 20:09	应用程序	171 KB
NEWS.txt	2015-04-27 19:26	文本文档	60 KB
Packman.exe	2005-02-20 6:04	应用程序	372 KB
Packman.map	2005-02-20 6:04	MAP 文件	682 KB
uninstall.exe	2024-07-09 20:35	应用程序	70 KB

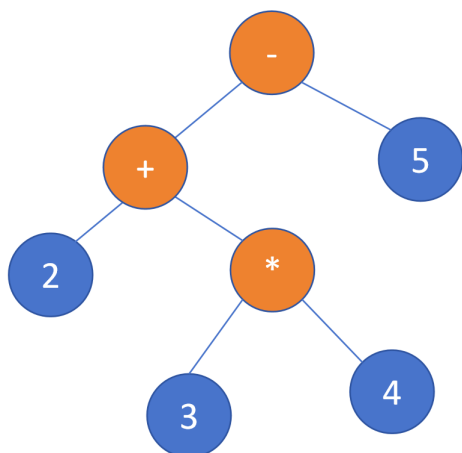
Dev-Cpp下有很多文件夹和文件，每个文件夹下也有很多的文件和文件夹。于是我们就可以画成下面的图：



👉如果将这张图倒过来，就像一颗树。

1.2 算式

如 `2+3*4-5`



👉上面也是一颗树，而且还是二叉树。

1.3 HTML文档

HTML文档是一个编写网页的文档，通过标签来控制页面的布局、组建的使用。在下面的代码中，`<html>` 就是一个根目录，`<h1>` 是的叶子。

```
1 <!DOCTYPE html>
2 <html lang="zh-CN">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-
6 scale=1.0">
7     <title>我的网页</title>
8     <style>
9         body {
10             font-family: Arial, sans-serif;
11         }
12         h1 {
13             color: #333;
14         }
15         p {
16             color: #666;
17         }
18     </style>
19 </head>
20 <body>
21     <header>
22         <h1>欢迎来到我的网页</h1>
23     </header>
```

```

23     <nav>
24         <ul>
25             <li><a href="#home">首页</a></li>
26             <li><a href="#news">新闻</a></li>
27             <li><a href="#contact">联系我们</a></li>
28         </ul>
29     </nav>
30     <section id="home">
31         <h2>首页内容</h2>
32         <p>这里是一些关于首页的描述性文字。</p>
33     </section>
34     <section id="news">
35         <h2>新闻</h2>
36         <p>这里是一些新闻内容。</p>
37     </section>
38     <section id="contact">
39         <h2>联系我们</h2>
40         <p>联系方式: example@example.com</p>
41     </section>
42     <footer>
43         <p>&copy; 2023 我的网页</p>
44     </footer>
45 </body>
46 </html>
47

```

1.4 .c 或者.cpp文档

这个文档也是树吗？其实和html页面一样，它的大括号也有层级。一级一级的都是

```

1  bool  isBracketRight(string str){
2      if(str.size() == 0){
3          return true;
4      }
5      stack<char> bracket;
6      int i = 0 ;
7      for(;i<str.size();i++){
8          if(str[i] == '('){
9              bracket.push(str[i]);
10             continue;
11         }
12         if(str[i] == ')'){
13             if(bracket.top() == '(') {
14                 bracket.pop();
15             }
16         }
17     }
18     if(bracket.empty()){
19         return true;

```

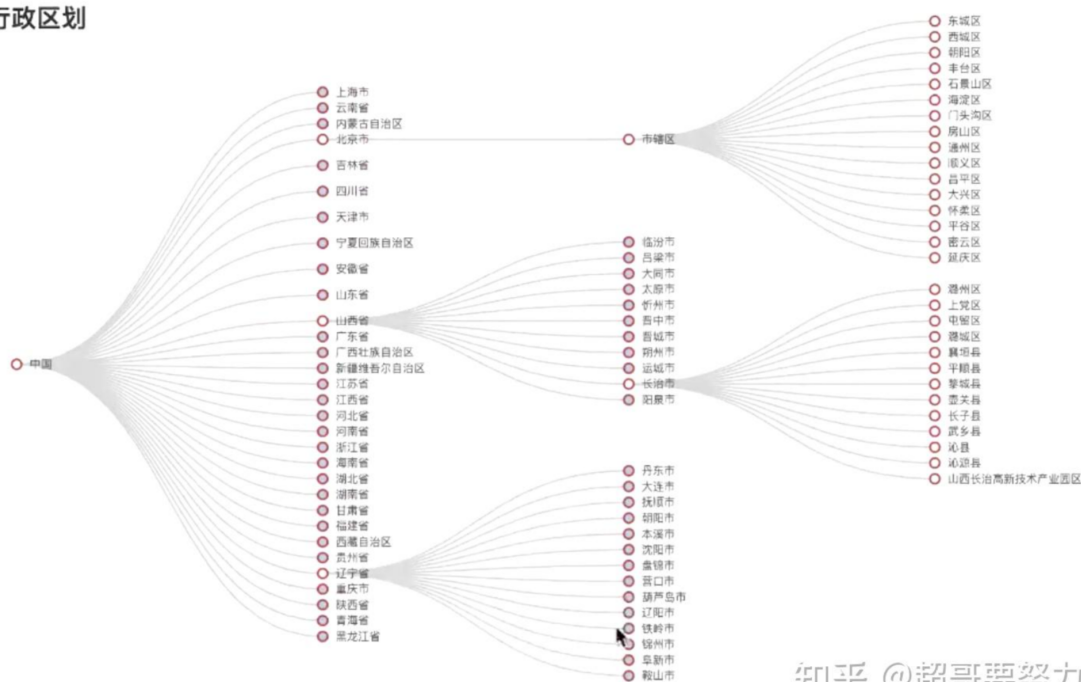
```

20     }
21     return false;
22 }

```

1.5 各级行政机构

中国行政区划



1.6 多级菜单



总之，凡是可以思维导图画出来的，都是树。

二、树的概念

2.1 基本定义

数据结构中的树是N个结点的有限集，是一种描述层次关系的非线性数据结构。其核心定义包含以下两个关键条件：

- 根结点唯一：在非空树中，有且仅有一个特定的称为根（Root）的结点，它没有前驱结点。
- 子树递归：当 $N > 1$ 时，其余结点可分为M个互不相交的有限集，每个集合本身又是一棵树，称为根的子树。

树结构直观上像一棵倒挂的树，根朝上叶朝下，结点间存在一对多的父子关系且无回路，广泛应用于文件系统及数据库索引等领域。

2.2 关于树的常用术语

- **度 (Degree)**：一个节点拥有的子节点个数；树的度是所有节点中最大的度。
- **叶节点 (终端节点)**：度为 0 的节点（无子节点）。
- **分支节点 (非终端节点)**：度大于 0 的节点。
- **兄弟节点**：具有相同父节点的节点。
-

高度/深度

:

- 节点的**深度**：从根到该节点的路径长度（边数）。
- 节点的**高度**：从该节点到最远叶节点的路径长度。
- 树的**高度**：根节点的高度。

2.3 树的特点

- **有且仅有一个根节点**：树的顶层节点，没有父节点。
- **除根节点外，每个节点有且仅有一个父节点**：确保了“一对多”的层次关系。
- **节点可有零个或多个子节点**：与二叉树（最多两个子节点）不同，普通树不限制子节点数量。
- **子树互不相交**：任意两个子树之间没有公共节点，否则构成图而非树。
- **总边数 = 节点数 - 1**：对于包含 n 个节点的树，恰好有 $n-1$ 条边连接节点。
- **递归具有**：树由根节点和若干互不相交的子树组成，每棵子树本身也是一棵树。
- **具有明确的层次关系**：从根（第1层）开始逐层向下，每个节点属于特定层次。
- **无环路**：从任一节点出发，沿父-子关系无法回到自身，避免循环依赖。

三、树的分类

树不像栈或队列，它并非线性的，而是二维的。二维的访问顺序（遍历）有很多的规律，由此诞生了很多种不同类型的树。一些树的访问非常复杂，也很烧脑。现阶段，我们不必依次学习这些树，因为这样做并不能提高编程能力。下面罗列一些树，其目的是让大家知道存在这些树。而我们最终所演示的仅仅是普通树和二叉树。

- **普通树 (General Tree)**：每个节点可以有任意多个子节点，无序或有序。
- **二叉树 (Binary Tree)**：每个节点最多有两个子节点，分别称为左子节点和右子节点，且左右顺序不可颠倒。
- **满二叉树 (Full Binary Tree)**：所有分支节点都有两个子节点，且所有叶子节点在同一层。

- **完全二叉树 (Complete Binary Tree)**：深度为 k 的二叉树，若其前 n 个节点与深度为 k 的满二叉树的前 n 个节点一一对应，则称为完全二叉树；堆通常基于此结构。
- **平衡二叉树 (Balanced Binary Tree)**：任意节点的左右子树高度差不超过 1，如 AVL 树。
- **二叉搜索树 (Binary Search Tree, BST)**：又称二叉排序树，左子树所有节点值小于根，右子树所有节点值大于等于根。
- **红黑树 (Red-Black Tree)**：自平衡二叉搜索树，通过颜色标记保证近似平衡，广泛用于语言标准库（如 C++ 的 `map`）。
- **B树与B+树 (B-Tree / B+ Tree)**：多路平衡搜索树，用于数据库和文件系统索引。
- **2-3树 (2-3 Tree)**：每个节点有 2 或 3 个子节点，所有叶子在同一层，属于绝对平衡树。
- **Trie树 (字典树)**：用于高效字符串检索，每个节点代表一个字符。
- **哈夫曼树 (Huffman Tree)**：带权路径最短的二叉树，用于数据压缩编码。
- **斜树 (Skewed Tree)**：所有节点只有左子树（左斜树）或只有右子树（右斜树）。

四、树的基本构造

普通树的构造如下。由于每个父节点下面的叶子个数是未知的，因此只记录第一个孩子节点的指针，其余孩子靠这个指针来记录。这种记录方式会产生冗余。

```
1 typedef struct Tree {
2     int child; // 子节点在顺序表中的位置
3     struct Tree *firstchild;
4 } Tree;
```

我认为，可以通过vector来构建一颗更简洁的树。

```
1 typedef struct Tree {
2     int child; // 子节点在顺序表中的位置
3     vector<Tree*> children;
4 } Tree;
```

我们不打算在使用链表来构建一颗树，而是通过数组（含动态数组）来构建一颗树，这也是目前最普遍的做法（可以AI查询：主流的构建普通树，是通过id和parent_id来实现吧）。具体做法是

1. 给每个节点一个id，树根节点id为0或者-1，
2. 每加入一个节点，id++（自增），从属的父节点用parent_id来实现。

具体的定义如下：

```
1 [{
2     "id": 0,
```

```

3         "parent_id": -1,
4         "data": "wangzhend"
5
6     },
7     {
8         "id": 1,
9         "parent_id": 0,
10        "data": "wangzhend"
11
12    },
13    .....
14 ]

```

五、特殊的树——二叉树

二叉树的每一根根节点，最多只能有两个叶子——一个算式就是一颗典型的二叉树。像红黑树等就是二叉树的一个子类。

二叉树的叶子节点数量十分明确，构建起来没那么复杂。用链表实现，代码如下：

```

1 typedef struct TreeNode {
2     int data;
3     struct TreeNode* left;
4     struct TreeNode* right;
5 } TreeNode;

```

5.1 二叉树具体的五种基本形态

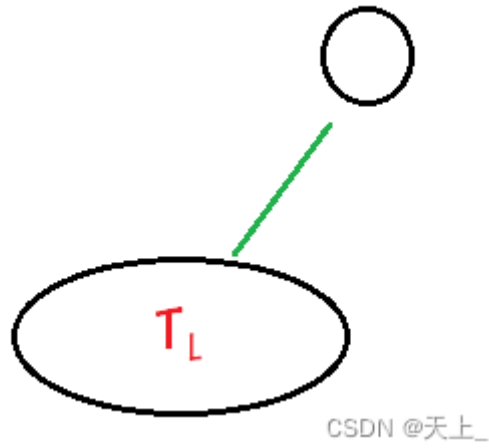
1.空树

Φ

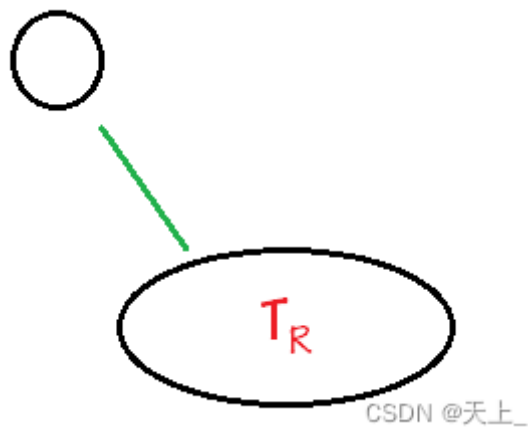
2.只有一个节点



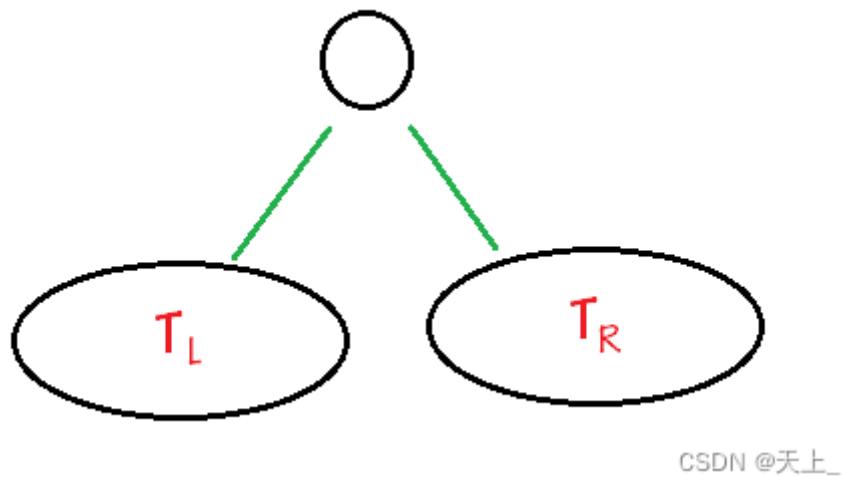
3.有左子树，但右子树为空



4.有右子树，但左子树为空



5.左右两子树都不为空

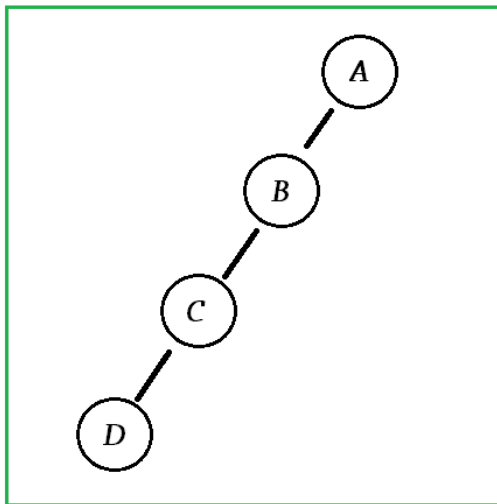


要注意，二叉树与普通的度为二的树不同的一点是：***二叉树的子树有左右顺序之分。***

5.2 特殊二叉树

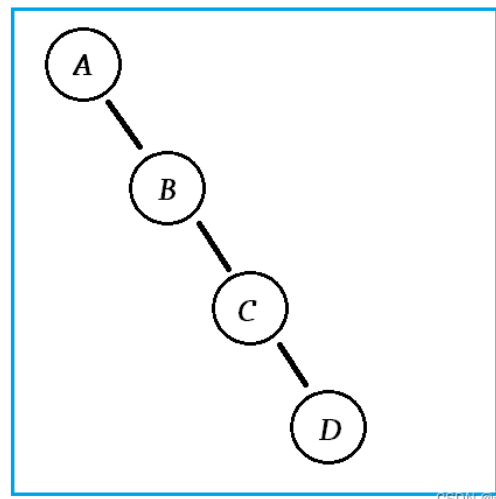
1.斜二叉树

Skewed Binary Tree



或

Skewed Binary Tree



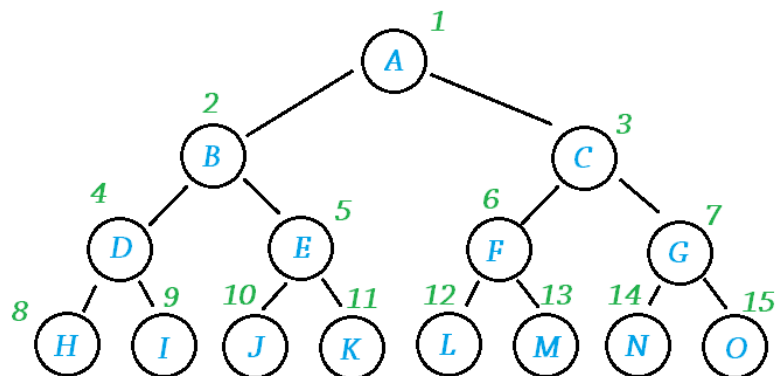
CSDN @天上_

斜二叉树都只有左儿子或者都只有右儿子，这样的二叉树，实际上相当于一个链表，形成了一个线性结构。

2. 满二叉树

又叫完美二叉树

Full Binary Tree
/ Perfect Binary Tree



CSDN @天上_

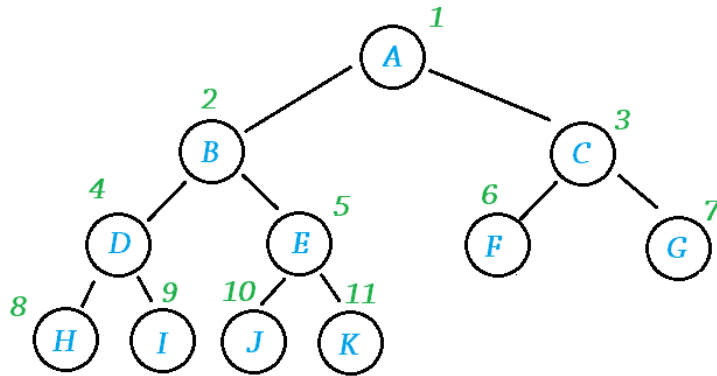
这样的二叉树是除了最底层的叶节点没有节点，其它每一个节点都有两个儿子；且叶节点都在同一层。

3. 完全二叉树

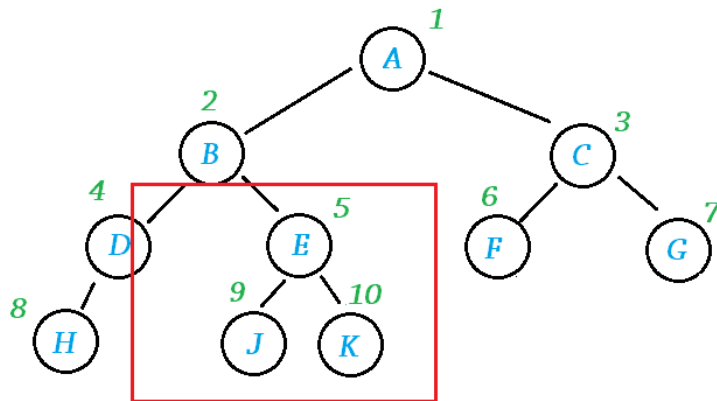
有 n 个节点的二叉树，对树中节点按从上至下、从左到右顺序进行编号，编号为 i ($1 \leq i \leq n$) 节点与满二叉树中编号为 i 节点在二叉树中位置相同。

简单地来说，完全二叉树的节点编号要与它满二叉树形态下的节点编号相一致，如下图：

Complete Binary Tree



Not Complete Binary Tree !!

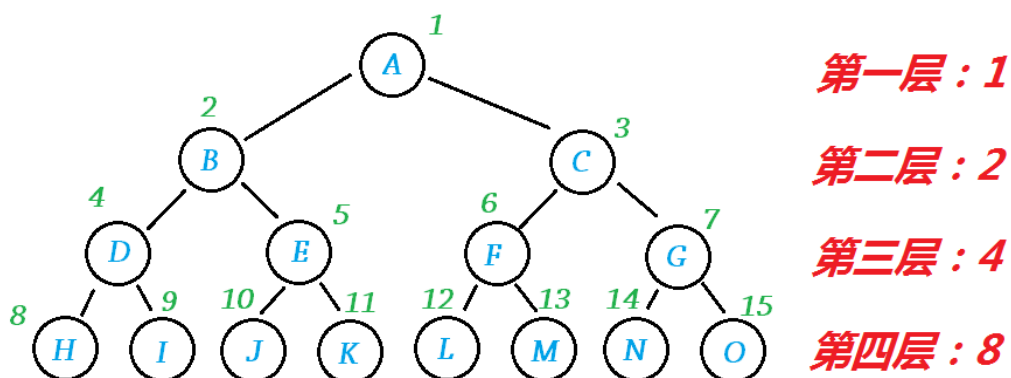


CSDN @天上_

5.3 二叉树的几个重要性质

- 一个二叉树第 i 层的最大节点数为 2^{i-1} , $i \geq 1$ 。

这个性质很好理解，像满二叉树这种树每一层都达到了最大节点数，节点数满足首项为1，公比为2的等比数列。



CSDN @天上_

- 深度为 K 的二叉树有最大节点总数为: $2^k - 1$, $k \geq 1$ 。

能达到最大节点数的树是怎样的呢？

当然就是完美二叉树啦，其节点数：

第一层：1

第二层: 2

第三层: 4

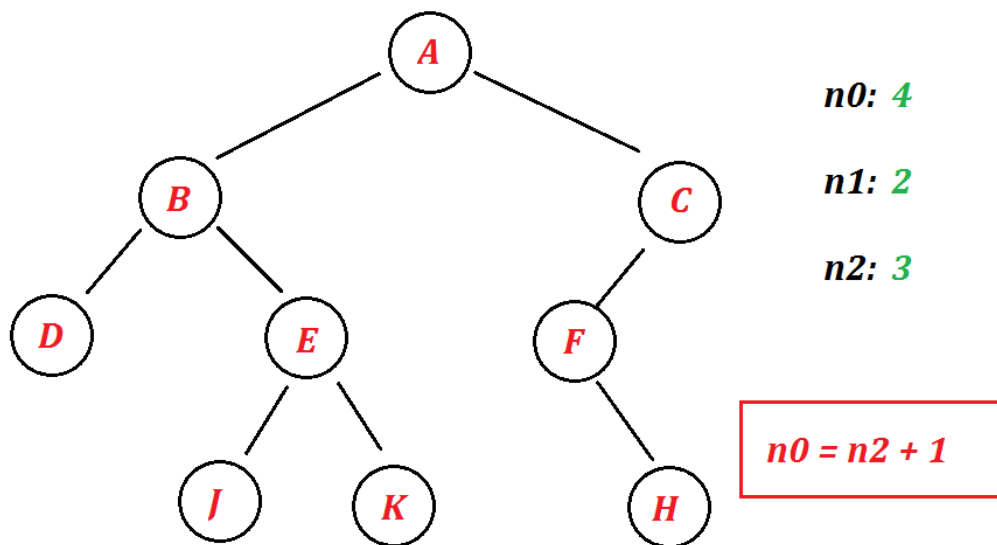
.....

第k层: 2^{k-1}

用等比数列求和公式 $S_n = \frac{a_1(1-q^n)}{1-q} (q \neq 1) = \frac{a_1-a_nq}{1-q} = \frac{a_1}{1-q} (1-q^n)$

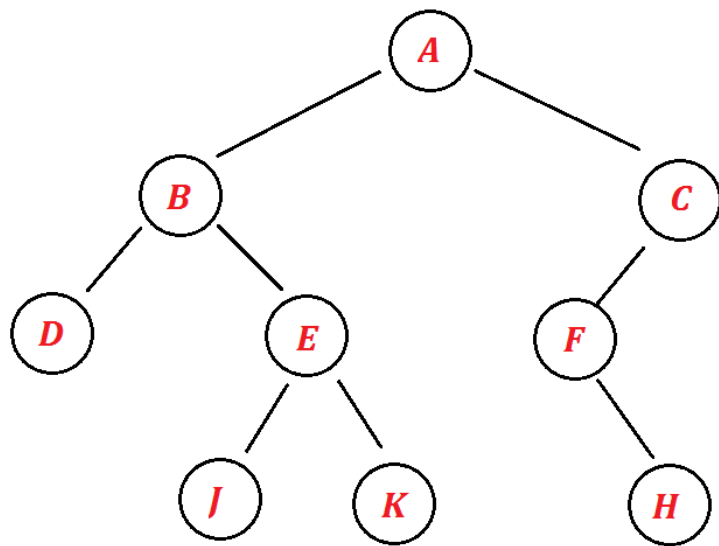
就可以求和最大节点数为: 2^k-1 。

- 对任何非空二叉树T, 若 n_0 表示叶节点的个数, n_2 是度为2的非叶节点个数, 那么两者满足关系* $n_0 = n_2 + 1$ 。 *



CSDN @天上

我们从边的角度来推导出这个性质:



从边的角度上看，这棵
二叉树的边为8条，节
点数为9.

所以可以把总的边数表
示为总的节点数减一
即：

$$n_0 + n_1 + n_2 - 1$$

再看节点，不同类型的节点，其下面的边数是不一样的。
比如 n_0 的节点下面一条边都没有， n_1 下面有一条边， n_2 下面有
两条边。

所以我们可以利用这个关系来表示总边数：

$$n_0 + n_1 + n_2 - 1 = n_0 * 0 + n_1 * 1 + n_2 * 2$$

化简整理过后，就得到了：

$$n_0 = n_2 + 1$$